Università degli Studi di Palermo

dipartimento di ingegneria unipa

Introduzione a Python

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

Eseguire Python: interprete interattivo e script

Due modalità di utilizzo dell'interprete

Ambiente interattivo (REPL)

Avvio: digitare python (o python3) nel terminale.

Terminale

```
$ python
Python 3.12.0 (main, ...)
>>> 2 + 3
5
>>> nome = 'Alice'
>>> print(f'Ciao, {nome}!')
Ciao, Alice!
>>> exit()          # oppure Ctrl+D / Ctrl+Z
```

■ Cos'è il REPL

Read → Eval → Print → Loop: ogni espressione viene letta, valutata e il risultato stampato immediatamente. Ideale per esplorare, testare snippet, ispezionare oggetti.

Esecuzione di uno script

Comando: python nomefile.py [argomenti]

saluto.py

```
# salva come saluto.py
import sys

nome = sys.argv[1] if len(sys.argv) > 1 else 'Mondo'
print(f'Ciao, {nome}!')
```

Terminale

```
$ python saluto.py
Ciao, Mondo!
$ python saluto.py Alice
Ciao, Alice!
```

Altre opzioni utili

- `python -c "print(42)"` — esegue una stringa come script
- `python -m modulo` — esegue un modulo installato (es. -m http.server)
- `python -i script.py` — entra nel REPL dopo l'esecuzione dello script

Tipi built-in principali

Dichiarazione, letterali e funzioni di conversione

Tipi scalari

Tipo	Letterale / Esempio	Note
`int`	42, -7, 0xFF, 0b1010	precisione arbitraria
`float`	3.14, 1e-3, float('inf')	IEEE 754 double (64 bit)
`complex`	2+3j, complex(1,-1)	parte reale + immaginaria
`bool`	True, False	sottoclasse di int (1/0)
`str`	'ciao', "hello", r'\n'	immutabile, Unicode
`bytes`	b'abc', bytes(4)	sequenza di byte immutabile
`NoneType`	None	valore nullo / assenza di valore

Tipi contenitore

Tipo	Letterale / Esempio	Caratteristiche
`list`	[1, 'a', 3.0]	ordinata, mutabile, duplicati ok
`tuple`	(1, 'a', 3.0) oppure 1,	ordinata, immutabile
`set`	{1, 2, 3} o set()	non ordinato, niente duplicati
`dict`	{'k': v, 1: [2,3]}	coppie chiave-valore, ordinato (3.7+)

Esempi di dichiarazione

Python

```python
# scalari
n   = 42
pi  = 3.14159
ok  = True
msg = 'Python 3'

# contenitori
lst = [10, 20, 30, 10]   # list: duplicati ok
tpl = (10, 20, 30)       # tuple: immutabile
st  = {10, 20, 30}       # set:  niente duplic.
d   = {'nome': 'Anna', 'eta': 25}

# verifica del tipo
type(n)       # <class 'int'>
isinstance(n, int)  # True
```

Conversioni esplicite

```python
int('42')       # 42
float('3.14')   # 3.14
str(100)        # '100'
bool(0)         # False   (0, None, [] → False)
list('abc')     # ['a', 'b', 'c']
tuple([1,2,3])  # (1, 2, 3)
set([1,1,2,3])  # {1, 2, 3}
dict(a=1, b=2)  # {'a': 1, 'b': 2}
```

Strutture di controllo del flusso

if / elif / else · for · while · match · gestione eccezioni

Selezione: if / elif / else

Python

```python
x = 7
if x < 0:
    print('negativo')
elif x == 0:
    print('zero')
else:
    print('positivo')  # stampa questo

# espressione condizionale (ternaria)
segno = 'pos' if x > 0 else 'non-pos'
```

Ciclo for (iterazione su sequenza)

```python
for i in range(5):         # 0,1,2,3,4
    print(i, end=' ')

for frutto in ['mela','pera','kiwi']:
    print(frutto)

# enumerate: indice + valore
for i, v in enumerate(['a','b','c']):
    print(i, v)             # 0 a, 1 b, 2 c

# break / continue / else
for n in range(10):
    if n == 5: break        # esce dal ciclo
    if n % 2: continue      # salta i dispari
else:
    print('completato')     # solo se no break
```

Ciclo while

Python

```python
n = 1
while n < 100:
    n *= 2
print(n)   # 128
```

Pattern matching: match / case (Python 3.10+)

```python
comando = 'quit'
match comando:
    case 'quit' | 'exit':
        print('Uscita')
    case 'help':
        print('Aiuto')
    case _:
        print('Comando sconosciuto')
```

Gestione eccezioni: try / except / finally

```python
try:
    x = int(input('Numero: '))
    print(10 / x)
except ValueError:
    print('Input non valido')
except ZeroDivisionError:
    print('Divisione per zero')
except Exception as e:
    print(f'Errore: {e}')
else:
    print('Ok')    # solo se nessuna eccezione
finally:
    print('Sempre eseguito')
```

Dichiarazione di funzioni

Parametri posizionali, keyword, *args, **kwargs, lambda

Sintassi base e parametri posizionali

Python

```
def saluta(nome, volte):
    """Stampa un saluto più volte."""
    for _ in range(volte):
        print(f'Ciao, {nome}!')

saluta('Anna', 2)     # posizionali (ordine conta)
saluta(volte=3, nome='Bob')  # keyword (ordine libero)
```

Valori di default e parametri opzionali

```
def potenza(base, esp=2, *, verbose=False):
    # parametri dopo * sono solo keyword
    risultato = base ** esp
    if verbose:
        print(f'{base}^{esp} = {risultato}')
    return risultato

potenza(3)              # 9
potenza(2, 10)          # 1024
potenza(2, verbose=True) # 2^2 = 4
```

Funzioni anonime (lambda)

```
quadrato = lambda x: x ** 2
quadrato(5)   # 25

# Utile con sorted, map, filter
dati = [('Bob',30),('Anna',25),('Luca',35)]
sorted(dati, key=lambda t: t[1])  # per età
```

*args e **kwargs: numero variabile di arg.

Python

```
def somma(*numeri):        # args: tupla
    return sum(numeri)

somma(1, 2, 3)    # 6
somma(1, 2, 3, 4) # 10

def mostra(**info):        # kwargs: dizionario
    for k, v in info.items():
        print(f'{k}: {v}')

mostra(nome='Anna', eta=25, citta='PA')

# Combinazione completa
def f(pos1, pos2, *args, kw1='x', **kwargs):
    pass
```

Unpacking in chiamata con * e **

```
def vettore(x, y, z):
    return x + y + z

coords = [1, 2, 3]
vettore(*coords)         # unpacking lista

params = {'x':1,'y':2,'z':3}
vettore(**params)        # unpacking dizionario
```

Ordine dei parametri

I parametri seguono questo ordine nella firma:
posizionali / *args / keyword-only / **kwargs
Un parametro / forza tutto ciò che viene prima ad essere solo posizionale.

La direttiva import

Moduli, pacchetti e gestione dei namespace

Forme di import

Python

```python
# 1. Import del modulo intero
import math
math.sqrt(16)      # 4.0  — accesso tramite namespace

# 2. Import con alias
import numpy as np
np.array([1,2,3])

# 3. Import di nomi specifici
from os.path import join, exists
join('/home', 'user')   # senza prefisso

# 4. Import di tutto (sconsigliato)
from math import *      # inquina il namespace!
sqrt(4)                 # 2.0  — ma da dove viene?
```

■ Evitare import *

from modulo import * può sovrascrivere nomi già definiti nel namespace corrente, rendendo il codice difficile da leggere. Preferire sempre import modulo o from modulo import nome.

Come funziona import

1. **Cerca** in sys.modules (cache); se già caricato riusa.
2. **Trova** il file: prima built-in, poi sys.path (dir. corrente, PYTHONPATH, stdlib, site-packages).
3. **Compila** il sorgente .py → bytecode .pyc (in __pycache__).
4. **Esegue** il codice del modulo dall'inizio alla fine (una sola volta).
5. **Lega** il nome nel namespace chiamante.

Ispezione dei moduli importati

Python

```python
import sys
print(sys.modules.keys())   # moduli in cache
print(sys.path)             # percorsi di ricerca

import math
dir(math)         # nomi esportati dal modulo
help(math.sqrt)   # documentazione
math.__file__     # percorso del file .py/.so
math.__name__     # 'math'
```

Pacchetti (package)

```python
# Struttura su disco:
# miopkg/
#   __init__.py   ← rende la dir un pacchetto
#   modulo_a.py
#   sotto/
#     __init__.py
#     modulo_b.py

import miopkg.modulo_a
from miopkg.sotto import modulo_b
```

Modulo come script: __main__

Comportamento diverso tra python file.py e import file

Come Python assegna __name__

Modalità	__name__ vale…
`python modulo.py`	'__main__' (eseguito direttamente)
`import modulo`	'modulo' (nome del file senza .py)
`python -m pacchetto`	'__main__' (entry-point del pacchetto)

Il pattern idiomatico

util.py

```
# util.py

def somma(a, b):
    return a + b

def _test_interni():
    assert somma(2, 3) == 5
    print('Test ok')

if __name__ == '__main__':
    # Questo blocco viene eseguito SOLO se il file
    # viene lanciato con: python util.py
    # Non viene eseguito con: import util
    _test_interni()
    print(somma(10, 20))
```

Dimostrazione del comportamento

python util.py (esecuzione diretta)

```
$ python util.py
Test ok
30
```

import util (in REPL o in altro modulo)

```
>>> import util
>>> util.somma(4, 5)
9
>>> # _test_interni() NON è stata chiamata
```

Perché è utile questo pattern?

- Permette di usare lo stesso file sia come libreria riutilizzabile sia come script autonomo.
- Il codice di test / demo non interferisce con chi importa il modulo.
- Consente di definire un entry-point chiaro per pacchetti complessi (__main__.py).

python -m e __main__.py

In un pacchetto, python -m miopkg cerca ed esegue miopkg/__main__.py — utile per tool da linea di comando.

Variabili di modulo utili

```
# accessibili sempre, sia in esecuzione che in import
__name__     # nome del modulo
__file__     # percorso del file sorgente
__doc__      # docstring di modulo (o None)
__package__  # nome del pacchetto contenitore
__spec__     # ModuleSpec (come è stato trovato)
```

Metodi speciali (dunder): rappresentazione e chiamata

__repr__, __str__, __call__, __doc__, __class__ e famiglia

__repr__ e __str__: come un oggetto si mostra

Python

```python
class Punto:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        # per sviluppatori: eval(repr(p)) == p (ideale)
        return f'Punto({self.x!r}, {self.y!r})'

    def __str__(self):
        # per utenti finali
        return f'({self.x}, {self.y})'

p = Punto(3, 4)
repr(p)      # "Punto(3, 4)"  ← usato dal REPL
str(p)       # "(3, 4)"       ← usato da print()
print(p)     # (3, 4)
f'{p}'       # chiama __str__  → '(3, 4)'
f'{p!r}'     # chiama __repr__ → 'Punto(3, 4)'
```

__call__: rendere un oggetto chiamabile

Python

```python
class Moltiplicatore:
    def __init__(self, fattore):
        self.fattore = fattore

    def __call__(self, x):
        return self.fattore * x

doppio = Moltiplicatore(2)
doppio(5)    # 10  ← l'oggetto si comporta come f()
doppio(21)   # 42

callable(doppio)  # True
# Le funzioni stesse implementano __call__:
callable(print)   # True
```

__doc__: documentazione integrata

```python
def somma(a, b):
    """Restituisce la somma di a e b."""
    return a + b

somma.__doc__      # 'Restituisce la somma di a e b.'
help(somma)        # stampa la docstring formattata

class Cane:
    """Classe che rappresenta un cane."""
    pass

Cane.__doc__   # 'Classe che rappresenta un cane.'
```

Fallback chain

Se __str__ non è definito, Python usa __repr__ come fallback.

Metodi speciali: operatori e protocolli

Aritmetica, confronto, contenitore, contesto

Operatori aritmetici e di confronto

Operatore	Metodo speciale
`a + b`	__add__(self, other) / __radd__
`a - b`	__sub__(self, other) / __rsub__
`a * b`	__mul__(self, other) / __rmul__
`a / b`	__truediv__ / __rtruediv__
`a // b`	__floordiv__
`a % b`	__mod__
`a ** b`	__pow__
`-a`	__neg__(self)
`abs(a)`	__abs__(self)
`a == b`	__eq__(self, other)
`a != b`	__ne__(self, other)
`a < b`	__lt__ / __le__ / __gt__ / __ge__
`a += b`	__iadd__ (operatori in-place)

Altri dunder importanti

Metodo	Quando viene chiamato
`__init__`	costruttore: p = Punto(3,4)
`__new__`	creazione dell'istanza (prima di __init__)
`__del__`	distruttore (garbage collection)
`__hash__`	hash(a) — per uso in set e dict
`__bool__`	bool(a), if a: …
`__bytes__`	bytes(a)
`__format__`	format(a, spec), f'{a:.2f}'
`__sizeof__`	sys.getsizeof(a)
`__class__`	type(a) — attributo, non metodo

Protocollo contenitore e sequenza

Operazione	Metodo speciale
`len(a)`	__len__(self)
`a[key]`	__getitem__(self, key)
`a[key] = v`	__setitem__(self, key, value)
`del a[key]`	__delitem__(self, key)
`x in a`	__contains__(self, item)
`iter(a)`	__iter__(self)
`next(a)`	__next__(self)
`reversed(a)`	__reversed__(self)

Protocollo context manager (with)

Python

```python
class Risorsa:
    def __enter__(self):
        print('apertura'); return self
    def __exit__(self, tipo, val, tb):
        print('chiusura')
        return False  # non sopprime eccezioni

with Risorsa() as r:
    print('uso')     # apertura → uso → chiusura
```

Riepilogo e Risorse

Python in una pagina

Argomenti trattati

- **REPL / script:** python (interattivo), python file.py, python -m
- **Tipi built-in:** int float complex bool str bytes | list tuple set dict
- **Controllo flusso:** if/elif/else · for · while · match · try/except/finally
- **Funzioni:** def, default, *, posiz., *args, **kwargs, lambda
- **import:** import m · from m import n · alias · sys.path · packages
- **__main__:** if __name__ == '__main__': isola codice di avvio
- **__repr__/__str__:** come l'oggetto si mostra nel REPL e con print()
- **__call__:** rende un oggetto chiamabile come una funzione
- **__doc__:** docstring — accessibile con help() e __doc__
- **Dunder operators:** __add__ __eq__ __len__ __getitem__ __enter__ …

Risorse ufficiali

- **Documentazione Python 3:** https://docs.python.org/3/
- **Tutorial ufficiale:** https://docs.python.org/3/tutorial/
- **Built-in Functions:** https://docs.python.org/3/library/functions.html
- **Data Model (dunder methods):** https://docs.python.org/3/reference/datamodel.html
- **Modulo sys:** https://docs.python.org/3/library/sys.html
- **importlib – sistema di import:** https://docs.python.org/3/reference/import.html
- **PEP 8 – stile del codice:** https://peps.python.org/pep-0008/
- **The Python Standard Library:** https://docs.python.org/3/library/index.html

Prossimi passi

→ Programmazione a oggetti: ereditarietà, polimorfismo, proprietà (@property)

→ Comprensioni: list/dict/set comprehension, espressioni generatore

→ Decoratori e gestori di contesto personalizzati

→ Librerie fondamentali: os, pathlib, json, csv, re, datetime

→ Ambienti virtuali: venv, pip, pyproject.toml